GAME PROGRAMMING COURSE — UNITY

ゲームでよく使うシステム

シーン遷移・サウンド・共有データ・PADコンフィグ

GAME SYSTEMS

対象：高等部2年（全4コマ）

LESSON 1

シーンのロード（フェード）

目標：画面を暗くしてからシーンを切り替える仕組みを作る

ゴール

シーン切り替えはどこで使う？

タイトル → ゲーム

スタートボタンを押したら、ふわっと暗くなってゲーム画面へ。

ステージクリア

クリア演出のあと、次のステージやリザルト画面へ。

リトライ

ゲームオーバーから同じステージを最初からやり直す。

完成したときの流れ

フェードアウト
画面が暗くなる

→

ロード
裏で次のシーンを読む

→

フェードイン
画面が明るくなる

悪い例

いろいろな所で直接 LoadScene する

C# あちこちのスクリプト

```
// TitleMenu.cs
SceneManager.LoadScene("Stage1");

// GoalFlag.cs
SceneManager.LoadScene("Stage2");

// GameOverMenu.cs
SceneManager.LoadScene("Stgae1");   // 打ち間違い…
```

POINT　それぞれのスクリプトが、シーン名の文字列を使って直接シーンを切り替えている。

悪い例の問題点

何が困るのか

① パッと切り替わる

前触れなく画面が変わる。演出がなく、プレイヤーが状況をつかみにくい。

② 読み込み中に固まる

LoadScene は読み込みが終わるまで処理が止まる。大きなシーンだと一瞬フリーズする。

③ 直す場所が多い

「フェードを入れたい」と思ったら、呼んでいる全スクリプトを直すことになる。

シーンの切り替えは、専用のクラス1か所にまとめる

基本

SceneManager の使い方

書き方	意味
using UnityEngine.SceneManagement;	SceneManager を使うために必要
SceneManager.LoadScene("名前")	読み込みが終わるまで待って切り替える
SceneManager.LoadSceneAsync("名前")	裏で少しずつ読み込む（非同期）
SceneManager.GetActiveScene().name	今のシーンの名前（リトライに使える）

> ⚠ 使うシーンは **File > Build Profiles > Scene List** に登録しておかないと読み込めない。

基本

LoadSceneAsync とコルーチン

C# 非同期ロードの基本形

```
IEnumerator LoadRoutine(string sceneName)
{
    AsyncOperation operation = SceneManager.LoadSceneAsync(sceneName);

    // 読み込みが終わるまで、1フレームずつ待つ
    while (!operation.isDone)
    {
        yield return null;
    }
}
```

AsyncOperation

読み込みの進み具合

isDone で完了したか、progress で進み具合（0〜1）がわかる。

yield return null;

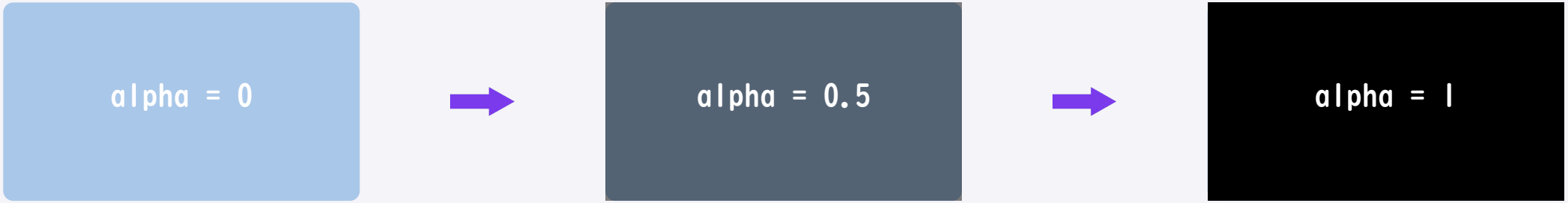
1フレーム待つ

待っている間もゲームは止まらない。フェードの描画も続けられる。

考え方

フェードのしくみ

画面全体をおおう**黒い画像（Image）**を置き、その**色の透明度（alpha）**を少しずつ変えるだけ。



使う部品	役割
Canvas	UI を表示する土台（Image を作ると自動でできる）
Image	画面いっぱい（1920 × 1080）の黒い画像。最初は完全に透明
Color.Lerp	2つの色の間を少しずつ変える（透明な黒 → 黒）

考え方

シーンをまたぐオブジェクト

シーンをロードすると、**前のシーンのオブジェクトはすべて消える。**
フェード用の Canvas まで消えると、暗いまま次のシーンへつなげられない。

SceneSingleton<T>

- シーンと一緒に消える
- カメラ・ステージの UI など

PersistentSingleton<T>

- シーンを切り替えても残る
- DontDestroyOnLoad を使う
- シーン遷移・サウンドなど

POINT SceneLoadManager は、ロード中も生き残る必要があるので **PersistentSingleton** を継承している。

考え方

シーン切り替えの手順

1	LoadScene(”名前”) を呼ぶ	どこからでも SceneLoadManager.Instance で呼べる
2	フェードアウト	isLoading を true にして、fadeImage を 透明 → fadeColor に変える
3	LoadSceneAsync	暗い間に、裏で次のシーンを読み込む
4	完了まで待つ	isDone が true になるまで yield return null
5	フェードイン	fadeColor → 透明 に戻し、isLoading で終わりを知らせる

提供スクリプト

SceneLoadManager.cs 全文（1/3）

C# SceneLoadManager.cs

```
using System.Collections;
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.SceneManagement;

public class SceneLoadManager : PersistentSingleton<SceneLoadManager>
{
    // フェード用
    [SerializeField] Image fadeImage;
    [SerializeField] float fadeTime = 1.0f;
    [SerializeField] Color fadeColor = Color.black;

    // 他スクリプトからロード中か確認する用
    public bool isLoading = false;

    /// <summary>
    /// ゲーム起動時に生成
    /// </summary>
    [RuntimeInitializeOnLoadMethod]
    static void Init()
    {
        SceneLoadManager pref = Resources.Load<SceneLoadManager>("Singletons/SceneLoadManager");
        if (pref != null) Instantiate(pref);
    }
```

提供スクリプト

SceneLoadManager.cs 全文（2/3）

C# SceneLoadManager.cs（続き）

```
26      /// <summary>
27      /// フェードしつつシーンをロードする処理
28      /// </summary>
29      /// <param name="sceneName"></param>
30      public void LoadScene(string sceneName)
31      {
32          StartCoroutine(_LoadScene(sceneName));
33      }
34
35      private IEnumerator _LoadScene(string sceneName)
36      {
37          isLoading = true;
38          Color baseColor = fadeImage.color;
39          float currentTime = 0;
40
41          // フェードアウト
42          while ((currentTime += Time.unscaledDeltaTime) < fadeTime)
43          {
44              float rate = currentTime / fadeTime;
45              Color nextColor = Color.Lerp(baseColor, fadeColor, rate);
46              fadeImage.color = nextColor;
47
48              yield return null;
49          }
```

提供スクリプト

SceneLoadManager.cs 全文（3/3）

C# SceneLoadManager.cs（続き）

```
51          // 非同期でシーンのロード
52          var async = SceneManager.LoadSceneAsync(sceneName);
53          while(!async.isDone)
54          {
55              // シーンロード中のフリーズ回避
56              // ロード進捗表示等をする場合はここで
57              yield return null;
58          }
59
60          // フェードイン
61          currentTime = 0;
62          while ((currentTime += Time.unscaledDeltaTime) < fadeTime)
63          {
64              float rate = currentTime / fadeTime;
65              Color nextColor = Color.Lerp(fadeColor, baseColor, rate);
66              fadeImage.color = nextColor;
67
68              yield return null;
69          }
70
71          isLoading = true;
72      }
73  }
```

実装

SceneLoadManager ① クラスと変数

C# SceneLoadManager.cs

```
1  using System.Collections;
2  using UnityEngine;
3  using UnityEngine.UI;
4  using UnityEngine.SceneManagement;
5
6  public class SceneLoadManager : PersistentSingleton<SceneLoadManager>
7  {
8      // フェード用
9      [SerializeField] Image fadeImage;
10     [SerializeField] float fadeTime = 1.0f;
11     [SerializeField] Color fadeColor = Color.black;
12
13     // 他スクリプトからロード中か確認する用
14     public bool isLoading = false;
```

PersistentSingleton<T>

シーンをまたいで残る。ロード中もフェード用の Image が消えない。

Image fadeImage

画面をおおう画像。Inspector で Image をドラッグして設定する。

fadeTime / fadeColor

Inspector で変えられる。白にすれば白フェード。

public bool isLoading

ロード中かどうか。他のスクリプトから確認できる。

提供スクリプトの解説

SceneLoadManager ② 起動時に自動で生成

C# SceneLoadManager.cs（続き）

```
16      /// <summary>
17      /// ゲーム起動時に生成
18      /// </summary>
19      [RuntimeInitializeOnLoadMethod]
20      static void Init()
21      {
22          SceneLoadManager pref = Resources.Load<SceneLoadManager>("Singletons/SceneLoadManager");
23          if (pref != null) Instantiate(pref);
24      }
```

[RuntimeInitializeOnLoadMethod]

起動時に呼ばれる

どのシーンから再生しても、最初に1回だけ実行される。

Resources.Load<T>("パス")

Resources から読む

Resources/Singletons/ の SceneLoadManager プレハブを読み込む。

if (pref != null)

見つかったときだけ生成

プレハブがないときは何もしない。生成すると Awake が呼ばれ、ずっと残る。

考え方

なぜシーンに置かずに自動で作るのか

シーンに置く方法

- 全部のシーンに置く必要がある
- 置き忘れると、そのシーンでは動かない
- 途中のシーンから再生するとエラーになる

起動時に自動で生成する方法

- どのシーンから再生しても、必ず1つある
- 置き忘れがない
- プレハブを直せば、全体に反映される

POINT そのため、作った Canvas は **Resources/Singletons/** にプレハブ化して、**シーン上からは消しておく。**

提供スクリプトの解説

SceneLoadManager ③ 外から呼ぶメソッド

C# SceneLoadManager.cs（続き）

```
26      /// <summary>
27      /// フェードしつつシーンをロードする処理
28      /// </summary>
29      /// <param name="sceneName"></param>
30      public void LoadScene(string sceneName)
31      {
32          StartCoroutine(_LoadScene(sceneName));
33      }
```

C# 使う側（タイトルのスクリプトなど）

```
1   // タイトルのスタートボタンを押したとき
2   SceneLoadManager.Instance.LoadScene("Stage1");
```

POINT　LoadScene は **StartCoroutine** でコルーチン _LoadScene を開始するだけ。使う側はコルーチンを意識しなくてよい。

実装

SceneLoadManager ④ フェードアウト

C# SceneLoadManager.cs（続き）

```
35      private IEnumerator _LoadScene(string sceneName)
36      {
37          isLoading = true;
38          Color baseColor = fadeImage.color;
39          float currentTime = 0;
40
41          // フェードアウト
42          while ((currentTime += Time.unscaledDeltaTime) < fadeTime)
43          {
44              float rate = currentTime / fadeTime;
45              Color nextColor = Color.Lerp(baseColor, fadeColor, rate);
46              fadeImage.color = nextColor;
47
48              yield return null;
49          }
```

isLoading = true

ここからロード中。他のスクリプトはこれを見て操作を止められる。

baseColor

最初の色（透明な黒）を覚えておく。最後に戻す色。

while の条件

時間を足してから fadeTime と比べる。「足す」と「比べる」を1行で書いた形。

Color.Lerp(a, b, rate)

rate が 0 なら a、1 なら b。透明 → fadeColor へ少しずつ変わる。

ポイント

なぜ unscaledDeltaTime を使うのか

C# deltaTime と unscaledDeltaTime

```
// deltaTime：ポーズ中（timeScale = 0）は 0 のまま進まない
currentTime += Time.deltaTime;

// unscaledDeltaTime：ポーズ中でも進む
currentTime += Time.unscaledDeltaTime;
```

Time.timeScale = 0

ポーズ中は時間が止まる

deltaTime だと、ポーズ画面から「タイトルへ」を押したときにフェードが進まない。

Time.unscaledDeltaTime

時間停止の影響を受けない

フェードやロードなど、ゲームの外側の演出はこちらを使う。

提供スクリプトの解説

SceneLoadManager ⑤ 非同期でロード

C# SceneLoadManager.cs（続き）

```
51          // 非同期でシーンのロード
52          var async = SceneManager.LoadSceneAsync(sceneName);
53          while(!async.isDone)
54          {
55              // シーンロード中のフリーズ回避
56              // ロード進捗表示等をする場合はここで
57              yield return null;
58          }
```

var async

型は AsyncOperation

var は右側から型を決めてくれる書き方。

async.isDone

読み込みが終わったか

終わるまで1フレームずつ待つので、画面が固まらない。

async.progress

進み具合（0〜1）

ロード画面のバーを作るなら、このループの中で使う。

提供スクリプトの解説

SceneLoadManager ⑥ フェードイン

C# SceneLoadManager.cs（続き）

```
60          // フェードイン
61          currentTime = 0;
62          while ((currentTime += Time.unscaledDeltaTime) < fadeTime)
63          {
64              float rate = currentTime / fadeTime;
65              Color nextColor = Color.Lerp(fadeColor, baseColor, rate);
66              fadeImage.color = nextColor;
67
68              yield return null;
69          }
70
71          isLoading = true;
72      }
```

POINT フェードアウトと同じ形で、Lerp の**はじめと終わりを逆**にしているだけ。fadeColor → baseColor（透明）に戻る。

準備

Unity で設定する ① Canvas と Image

手順	作業	ポイント
1	Hierarchy で Create ＞ UI ＞ Image	Canvas も一緒に作られる
2	Image のサイズを 1920 × 1080 にする	画面全体をおおう
3	Image の Color を黒、透明度（A）を 0 にする	最初は完全に透明（baseColor になる）
4	生成された Canvas に SceneLoadManager をアタッチ	Image ではなく Canvas に付ける
5	Canvas の名前を SceneLoadManager にする	Resources.Load の名前と同じにする
6	Canvas の Graphic Raycaster をオフにする	透明な Image がボタンのクリックをさえぎらないように
7	Canvas の Sort Order を最大（32767）にする	どの UI よりも手前にフェードを表示する
8	Fade Image 欄に Image をドラッグ	fadeImage に設定される

準備

Unity で設定する ② プレハブ化と登録

手順	作業	ポイント
9	Assets/Resources/Singletons/ フォルダを作り、Canvas をドラッグしてプレハブ化	フォルダ名・場所を間違えない
10	シーン上の SceneLoadManager を削除	起動時に自動で生成されるため
11	使うシーンを File ＞ Build Profiles ＞ Scene List に登録	登録しないと読み込めない

POINT **Resources** は特別な名前のフォルダ。中のファイルは `Resources.Load` で名前を指定して読み込める。

演習

シーン遷移を作り込もう

1 ボタンからシーンを切り替える

タイトルのボタンを押したら、Stage1 にフェードで切り替わるようにしよう。

ヒント：`SceneLoadManager.Instance.LoadScene("Stage1");`

2 フェードの時間と色を変える

プレハブの Fade Time・Fade Color を変えて、白フェードにしてみよう。

3 連打してもロードを1回にする

isLoading を使って、ロード中はボタンを押しても LoadScene しないようにしよう。

ヒント：`if (SceneLoadManager.Instance.isLoading) return;`

4 発展：ロードの進み具合を表示する

読み込みの進み具合をバーで表示しよう。

ヒント：progress は 0.9 で止まるので 0.9 で割る